EERoom

Browser-Based Electrical Engineering Simulation Platform

Final Architecture Plan (Design Specification)

Author	Aiden Vargas
Course Number	CST-452 Senior Project II
Instructor	Professor Thompson
Revision	2.0 (final, supersedes 1.1 from Milestone 3)
Date	November 4, 2026
Live Application	https://www.eeroom.io
Source Repository	https://github.com/AidenVargas03/EERoom

Grand Canyon University | College of Engineering and Technology

Abstract

EERoom is a browser-based electrical engineering simulation platform with three tools: an Ohm's Law and power calculator, a waveform visualizer that demonstrates aliasing, and a logic gate sandbox that generates truth tables and Boolean expressions. This final revision of the Architecture Plan describes the system as it was built and deployed at the end of the third code iteration, replacing the planned design from Milestone 3 wherever the implementation differs. It contains the final technical requirements, the system and deployment architecture, the module design, the database schema and data dictionary, every API endpoint, sequence diagrams for the main flows, the algorithms, the screen specifications with screenshots of the live application, the security design, and a traceability table from requirements to modules. Section 2.4 lists every change from the Milestone 3 design and the reason for it.

Revision and Signoff Sheet

Date	Author	Revision	Notes
CST-451	Aiden Vargas	1.0	Initial architecture plan submitted for Milestone 3
CST-451	Aiden Vargas	1.1	Scope refined to three tools; circuit simulator, Google OAuth, and Stripe removed
11/04/2026	Aiden Vargas	2.0	Final: updated to the built system after three code iterations (Milestone 6)

Instructor Feedback

Project approval	☐ Professor Thompson
Submitted by	Aiden Vargas, 11/04/2026

Contents

1. Final Technical Requirements
2. Design Planning Summary
3. Overview of Design Concepts
4. Detailed Solution Architecture
    4.1 System Architecture Overview
    4.2 Physical Architecture Diagram
    4.3 Module and Class Design
    4.4 Entity Relationship Diagram
    4.5 Database Schema
    4.6 API Endpoint Definitions
    4.7 Sequence Diagrams
    4.8 Algorithm Descriptions
    4.9 Screen Interface Specifications
    4.10 Security Design
5. Data Dictionary
6. Requirements Traceability
Appendix A: References

1. Final Technical Requirements

The technical requirements below are the ones the finished system actually uses. Konva.js and math.js from the Milestone 3 plan are no longer listed: the logic board is drawn in SVG on a grid (SC-04) and the Ohm's Law calculator needs only arithmetic. Three were added during development.

ID	Requirement	Final Solution	Status
TR-01	Frontend framework	React 18 with hooks, React Router 6 for routing	Used
TR-02	Drawing for the tools	Inline SVG for the logic board; HTML5 Canvas for the waveform display	Changed (was Konva.js)
TR-03	Math engine	Plain JavaScript in pure functions (waveform.js, logicEvaluator.js, Ohm's Law branching)	Changed (was math.js)
TR-04	Backend API server	Node.js with Express 4 on Railway	Used
TR-05	Relational database	PostgreSQL 15 on Supabase with a JSONB column for tool state	Used
TR-06	Authentication	Supabase Auth (bcrypt) plus EERoom's own 24-hour JWT (jsonwebtoken)	Used
TR-07	Frontend hosting	Vercel with automatic HTTPS and deploys from GitHub	Used
TR-08	Backend hosting	Railway container with automatic deploys from GitHub	Used
TR-09	CSS framework	Tailwind CSS 3 with a shared palette in theme.js	Used
TR-10	Version control	Public GitHub repository, branch main	Used
TR-11	Development runtime	Node.js and npm with the Vite 5 dev server	Used
TR-12	Rate limiting	express-rate-limit on /auth: 10 requests per 15 minutes per address	Added
TR-13	Automated testing	Vitest 1.6 for unit tests; Supertest 7 for API tests	Added
TR-14	Custom domain	www.eeroom.io pointed at Vercel	Added

2. Design Planning Summary

2.1 Project Overview

EERoom gives students free, browser-based access to three electrical engineering simulation tools. Anyone can use the tools without an account. A free account adds saving, a dashboard of saved projects, password recovery, and read-only share links.

2.2 Situation and Need

Simulation software such as MATLAB, Ansys, and Cadence is licensed per seat, installed on lab computers, and slow to learn, which keeps it out of reach for many students working on their own. EERoom covers the fundamentals that appear early in an electrical engineering program with tools that open in any browser.

2.3 Solution and Rationale

The system is a three-tier web application: a React client, an Express REST API, and PostgreSQL with Supabase Auth. Every component is free or low cost and well documented. All simulation math runs in the browser, so results are instant and the server only stores and returns projects.

2.4 Design Changes Since Milestone 3

ID	Change	Reason
SC-01	DC circuit simulator removed	Highest-risk feature; could not be finished and tested in the time available
SC-02	Google OAuth removed	Email and password login meets the requirement
SC-03	Stripe, Pro tier, save limit, and PDF export removed	Not needed to show the platform works; removes a payment security surface
SC-04	Grid placement in SVG instead of react-konva drag and drop	Predictable wire endpoints, easier to use and demonstrate; konva, react-konva, and mathjs removed
SC-05	Ideal-versus-sampled overlay instead of three overlaid signals	Teaches aliasing and keeps saved data simple
SC-06	Guest access, deferred at Milestone 4	Built in iteration 3: tools are public; saving still needs an account
SC-07	Password reset form, deferred at Milestone 4	Built in iteration 3: Forgot and Reset Password pages and POST /auth/update-password
SC-08	Vitest instead of Jest for API tests	One test runner for the client and the server
Added	Express app split into app.js and index.js	Lets the API tests import the app without opening a port
Added	Public /portfolio page	Internet-facing project portfolio required for Milestone 6

3. Overview of Design Concepts

3.1 High-Level System Design

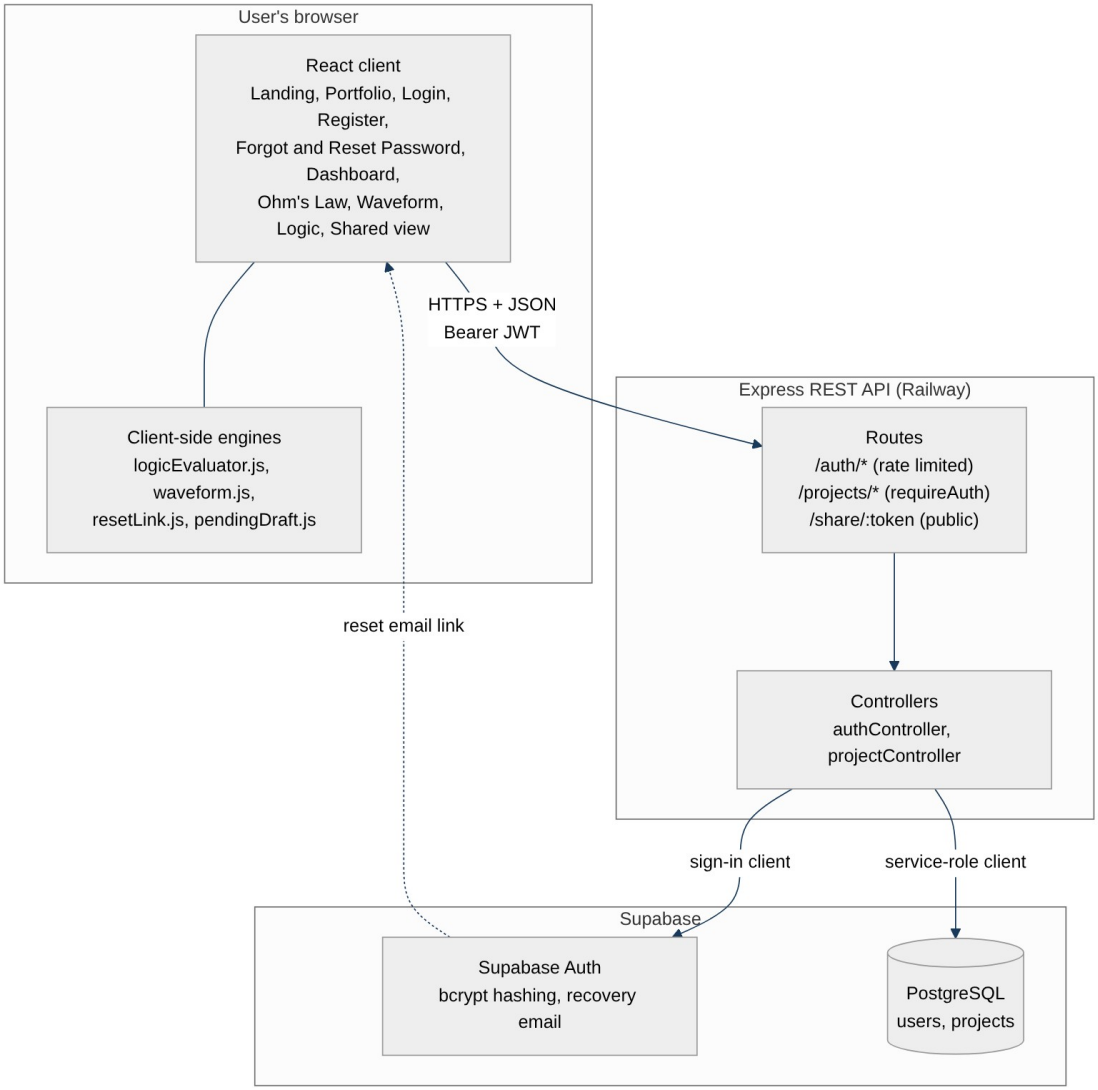


Figure 1. System architecture: the React client, the Express API, and Supabase

3.2 Process Flow

A visitor lands on the home page and can open any tool immediately. Results are calculated in the browser as they type. When a guest chooses to save, the tool stores its state in session storage and sends them to sign up or log in; after login, that work is saved to the new account and reopened. A logged-in user's projects appear on the dashboard, newest first, and reopen with their saved state. Sharing a project creates a random 64-character token, and anyone with the link can view the project read-only without an account. A user who forgets their password requests a recovery email and sets a new password from the link in it.

3.3 Design Concept Approval Statement

As the developer and sole stakeholder of EERoom, I have reviewed this final design against the implemented system and confirm that it describes the application as built and deployed, and that it meets the functional and non-functional requirements in the final Requirements Specification.

Developer signature and date	Aiden Vargas, 11/04/2026

4. Detailed Solution Architecture

4.1 System Architecture Overview

Presentation tier: a React single page application served from Vercel at www.eeroom.io. It contains every page, the shared components, and the client-side engines (logicEvaluator.js, waveform.js, resetLink.js, pendingDraft.js). Application tier: an Express REST API on Railway with three route groups. The /auth routes are public and rate limited, the /projects routes require a valid JWT, and GET /share/:token is public and read-only. Data tier: Supabase hosts PostgreSQL with the users and projects tables and provides authentication, bcrypt password hashing, and the recovery email. The API uses two Supabase clients: a service-role client for data and a separate client for sign-in and recovery, so a user sign-in can never change the session used for database queries.

4.2 Physical Architecture Diagram

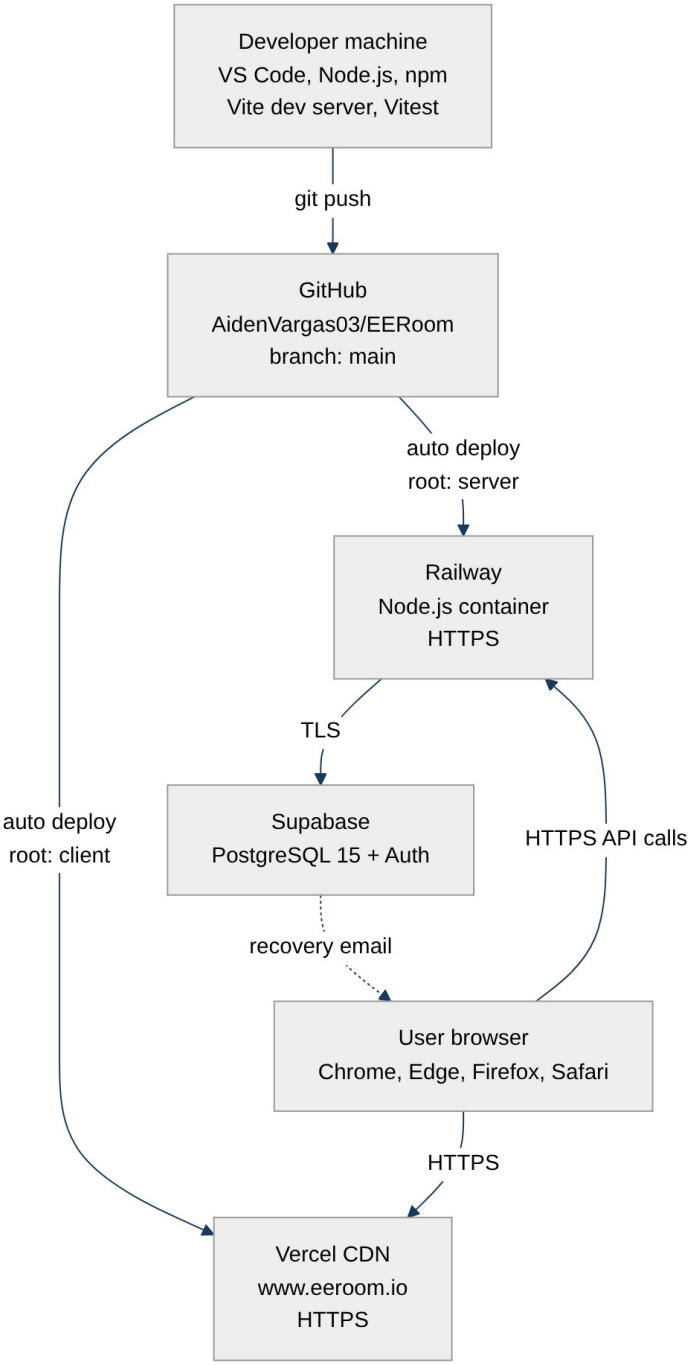


Figure 2. Deployment: GitHub deploys the client to Vercel and the API to Railway on every push to main

4.3 Module and Class Design

The client is organized as modules rather than classes. Figure 3 shows the main modules, the functions each exposes, and how they depend on one another.



Figure 3. Module design of the client and server

Module	Location	Responsibility
useAuth	client/src/hooks/useAuth.jsx	Holds the token and user in React context and localStorage; register, login, logout
projects API	client/src/api/projects.js, client.js	Axios calls to /projects; attaches the JWT to every request
Tool pages	client/src/pages/*ToolPage.jsx	Inputs, drawing, save and reopen, guest mode

Module	Location	Responsibility
logicEvaluator	client/src/utils/logicEvaluator.js	Gate definitions, validation, evaluation, truth table, expressions
waveform	client/src/utils/waveform.js	Signal shapes, sampling, RMS, range, Nyquist check
resetLink	client/src/utils/resetLink.js	Reads the recovery token or error from a reset link
pendingDraft	client/src/utils/pendingDraft.js	Holds a guest's work in session storage until login
requireAuth	server/middleware/authMiddleware.js	Verifies the Bearer JWT and sets req.user
authController	server/controllers/authController.js	register, login, resetPassword, updatePassword
projectController	server/controllers/projectController.js	Project CRUD and share token creation, each filtered by owner
app / index	server/app.js, server/index.js	Builds the Express app (CORS, JSON, routes, errors); starts the server

4.4 Entity Relationship Diagram

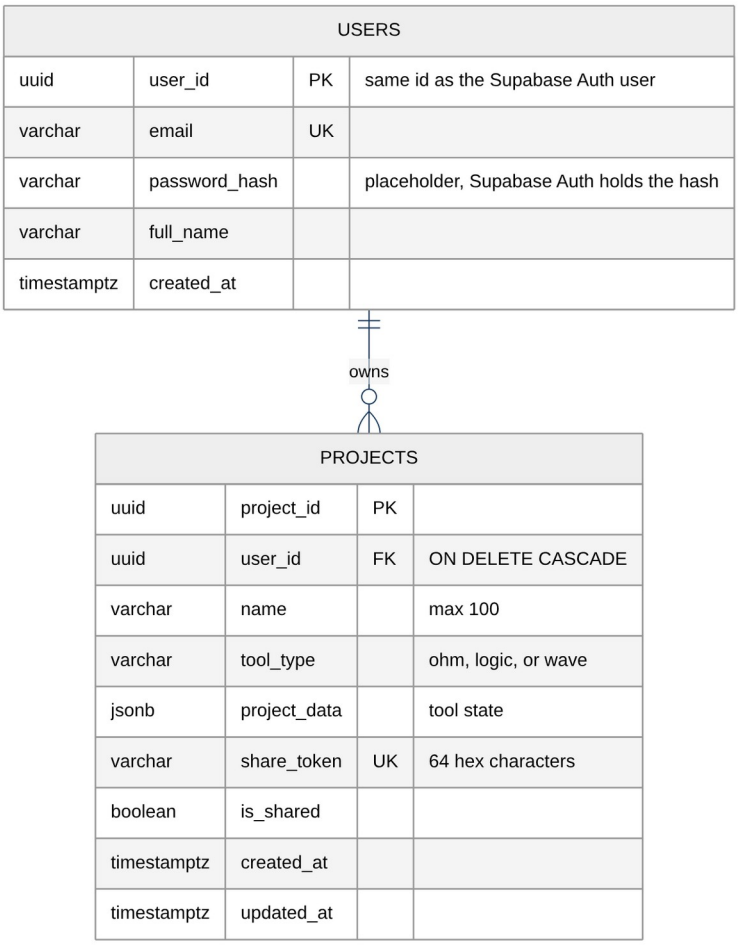


Figure 4. One user owns many projects; deleting a user deletes their projects

4.5 Database Schema

From server/db/schema.sql. Row Level Security is enabled on both tables with no policies, so only the server's secret key can read or write them.

```
CREATE TABLE users (
  user_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name     VARCHAR(100),
  created_at    TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE projects (
  project_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  name          VARCHAR(100) NOT NULL,
  tool_type     VARCHAR(20) NOT NULL CHECK (tool_type IN ('ohm','logic','wave')),
  project_data  JSONB NOT NULL,
  share_token   VARCHAR(64) UNIQUE,
  is_shared     BOOLEAN NOT NULL DEFAULT FALSE,
  created_at    TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at    TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_projects_user_id ON projects(user_id);
```

4.6 API Endpoint Definitions

Method and Path	Access	Request	Response
POST /auth/register	Public, rate limited	email, password, fullName	201; 400 missing or duplicate; 500 with rollback
POST /auth/login	Public, rate limited	email, password	200 token and user; 401 Invalid email or password
POST /auth/reset-password	Public, rate limited	email	200, the same message for every email
POST /auth/update-password	Public, rate limited	accessToken, password (8+)	200; 400 short or missing; 401 bad or expired token
GET /projects	JWT	None	200, the caller's projects, newest first
POST /projects	JWT	name, tool_type, project_data	201 project; 400 invalid
GET /projects/:id	JWT, owner only	None	200 project; 404 if missing or not the owner's
PUT /projects/:id	JWT, owner only	name and/or project_data	200 project; 404
DELETE /projects/:id	JWT, owner only	None	200; 404
POST /projects/:id/share	JWT, owner only	None	200 shareToken (reused if already shared); 404

Method and Path	Access	Request	Response
GET /share/:token	Public	None	200 name, tool_type, project_data, created_at; 404
GET /health	Public	None	200 status ok

Every /auth route shares one limiter of 10 requests per 15 minutes per address, which returns 429 when exceeded. Errors are always returned as JSON with an error field.

4.7 Sequence Diagrams

4.7.1 Login, Including a Guest's Pending Work

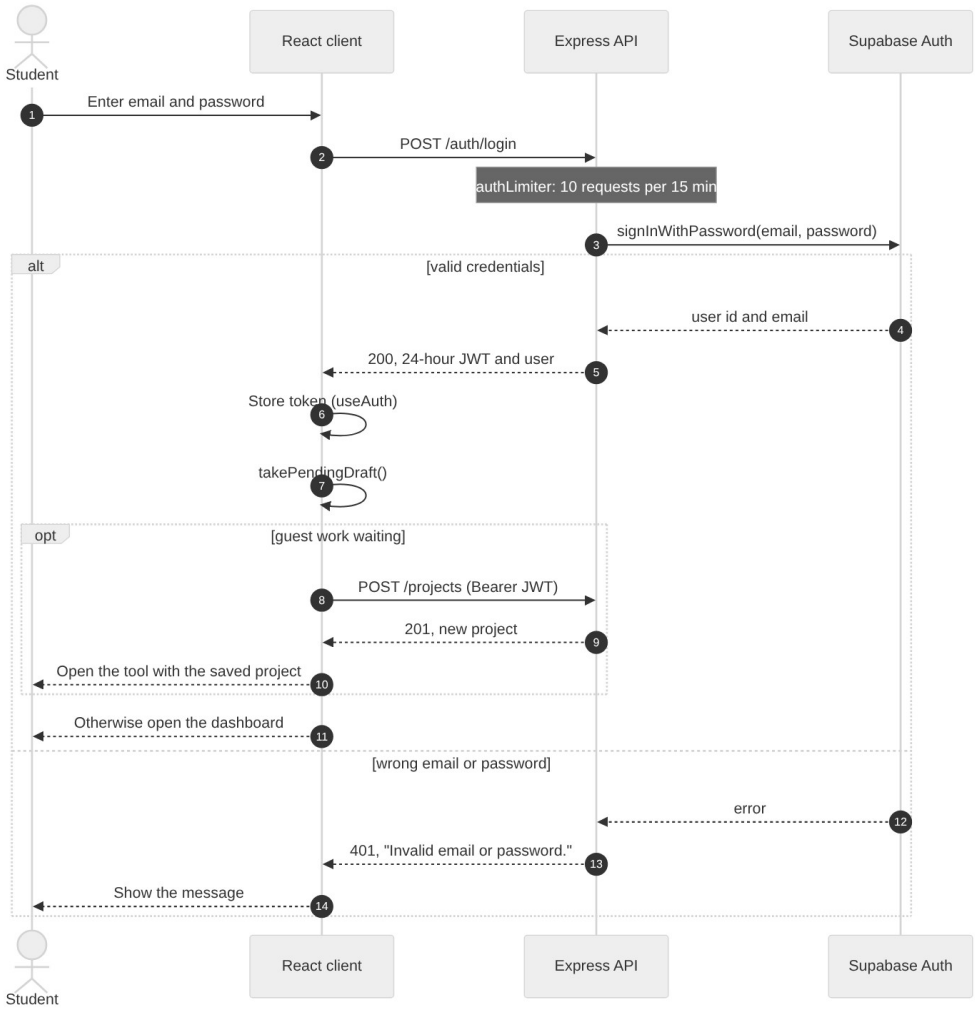


Figure 5. Login sequence

4.7.2 Save Project

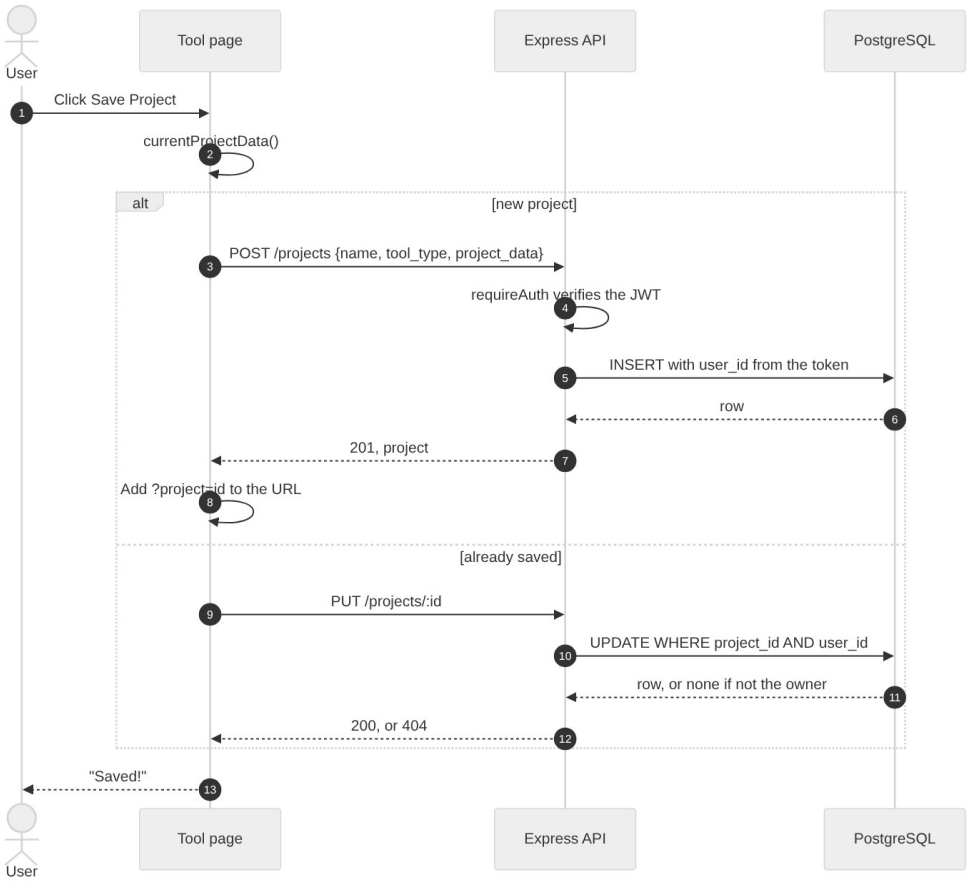


Figure 6. Save sequence: create on the first save, update after that

4.7.3 Password Reset

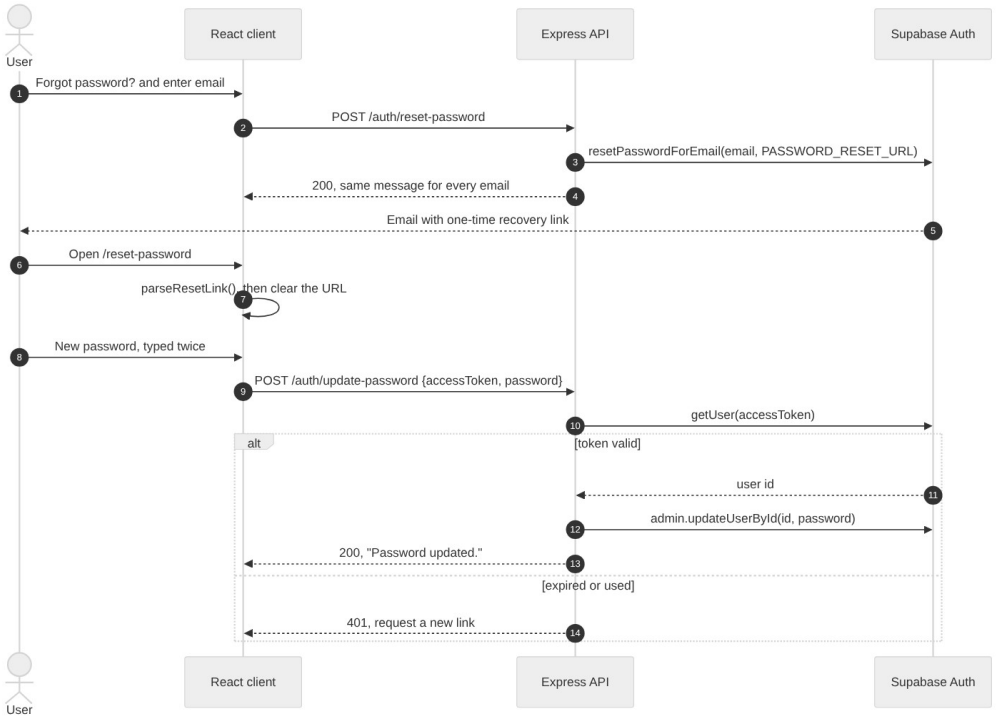


Figure 7. Password reset sequence

4.7.4 Share and View a Project

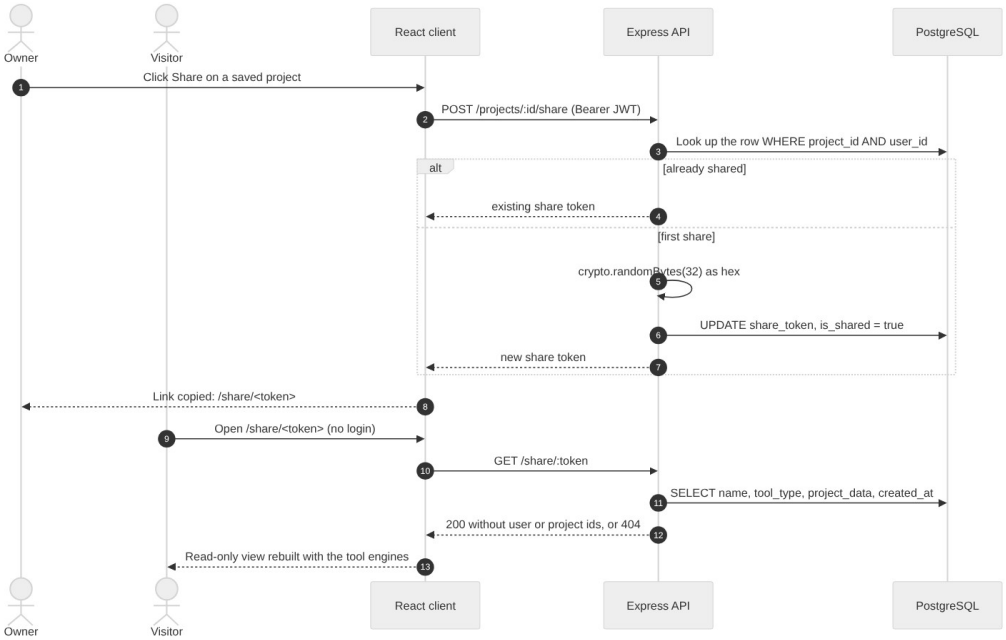


Figure 8. Share link creation and public viewing

4.8 Algorithm Descriptions

4.8.1 Ohm's Law Solve-For

The page remembers the two fields the user edited most recently. The third field is the one solved for: R = V / I, I = V / R, or V = I × R. Power is P = V × I. Solving for R with I = 0, or for I with R = 0, shows an error instead of dividing by zero. When a saved project loads, all three values are shown as saved until the user edits a field.

4.8.2 Boolean Circuit Evaluation

- The circuit is a directed graph of nodes (INPUT, the seven gates, OUTPUT) and edges that record which input pin (toPort) each wire lands on.
- validateCircuit() checks the structure: known node types, edges that point at real nodes, and pin numbers in range.
- evaluateCircuit() evaluates each output with a depth-first search from that output back to the inputs, caching each node's value. A node seen again while still on the current path means a feedback loop, and the evaluator throws an error instead of guessing.
- An input pin with no wire reads as logic 0 and is returned in a floating list, so the page can mark it.
- generateTruthTable() evaluates every combination of the inputs, 2^n rows, and refuses more than 10 inputs (1,024 rows).
- toExpression() walks the same graph to write each output as a nested expression, such as Y = (A XOR B).

4.8.3 Waveform Sampling and Aliasing

sampleAt() returns the value of a sine, square, sawtooth, or triangle wave at time t from the amplitude, frequency, phase, and DC offset. The display draws the ideal curve densely, then overlays the points a chosen sample rate would record. isAliased() returns true when the sample rate is at or below twice the signal frequency (the Nyquist rate); undersampling then shows a false wave at |f - fs|, which the unit tests check. rms() and voltageRange() produce the readouts.

4.8.4 Share Tokens

A share token is crypto.randomBytes(32) written as 64 hexadecimal characters, which cannot be guessed. Sharing a project that is already shared returns its existing token, so links already sent keep working.

4.8.5 Guest Work Carried Through Login

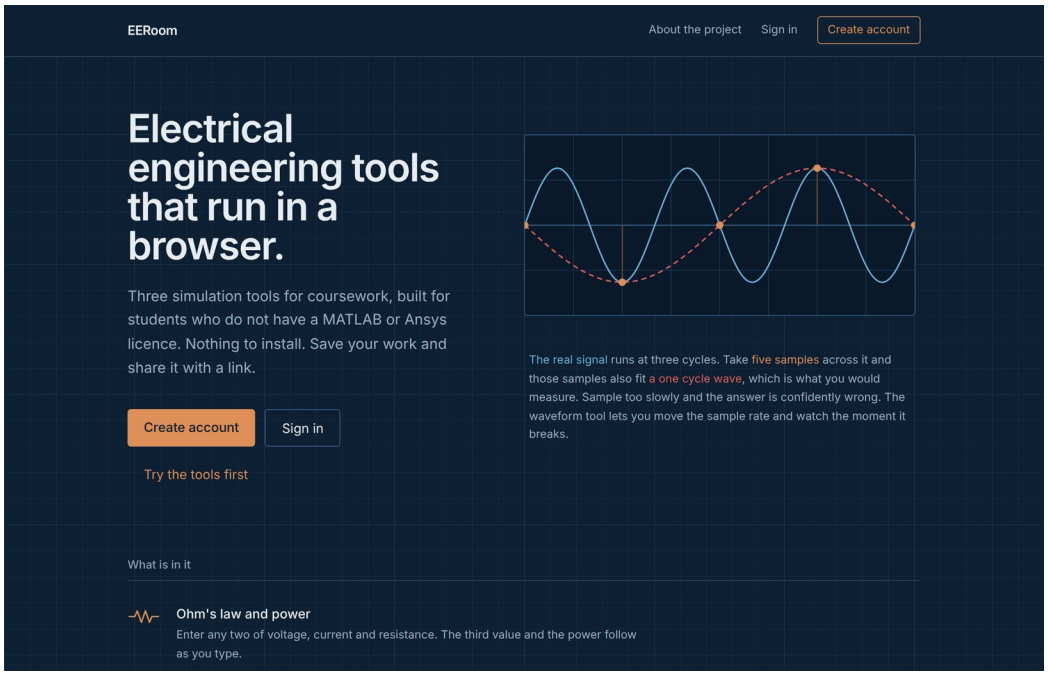
When a guest presses Sign up to save or Log in to save, the tool page stores { toolType, name, projectData } in session storage. After a successful login, takePendingDraft() reads and deletes it in one step, the login page creates the project through the API, and the tool opens with it. Invalid drafts are discarded, and a draft can never be saved twice.

4.9 Screen Interface Specifications

Screenshots are of the application running from the final source code. The dashboard and shared view show sample projects.

4.9.1 Landing Page

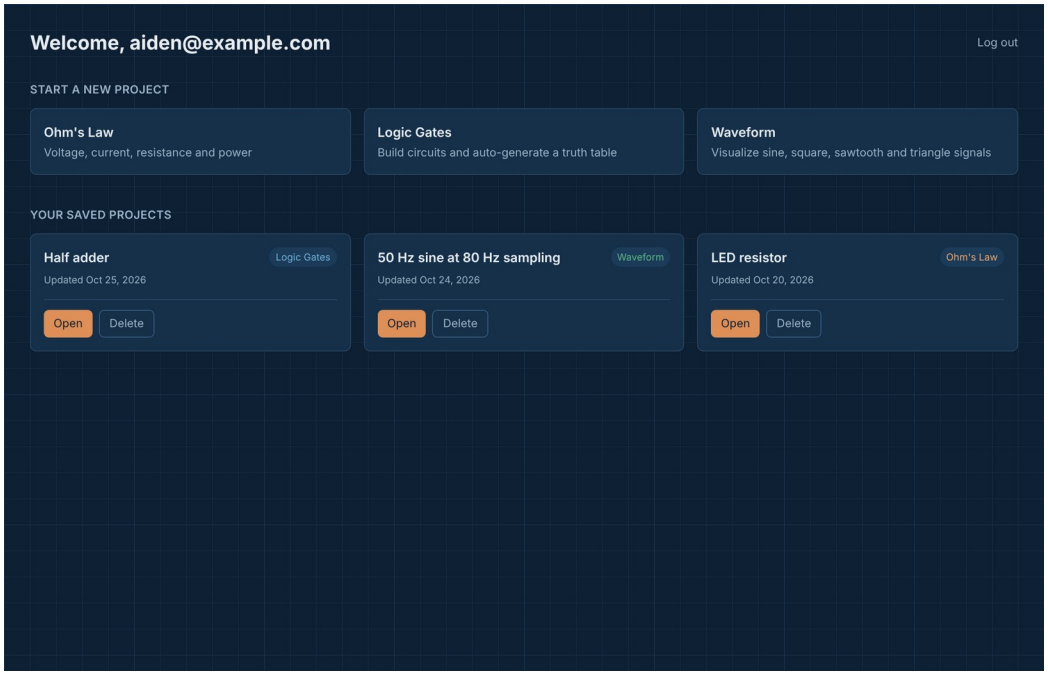
Headline, an accurate aliasing illustration, Create account, Sign in, and Try the tools first, then the three tools with Try it now links.



Landing Page

4.9.2 User Dashboard

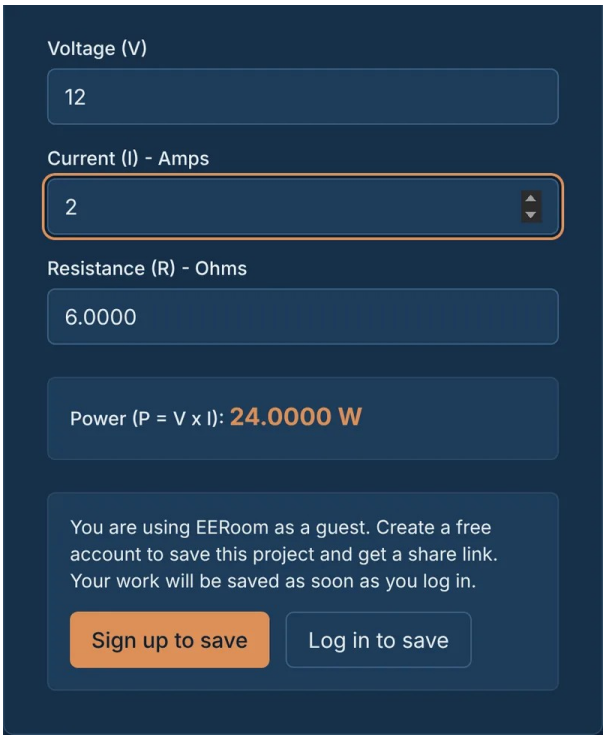
Quick-start cards for each tool and the user's saved projects, newest first, with Open and Delete (with an inline confirmation).



User Dashboard

4.9.3 Ohm's Law Calculator

Three inputs, the power readout, and, for a guest, the Sign up to save and Log in to save prompt in place of Save.



Ohm's Law Calculator

4.9.4 Logic Gate Sandbox

Component palette, 7 by 5 grid board that scales to fit tablets, wires that turn copper when carrying a 1, Boolean expressions, and the truth table.



Logic Gate Sandbox

4.9.5 Waveform Visualizer

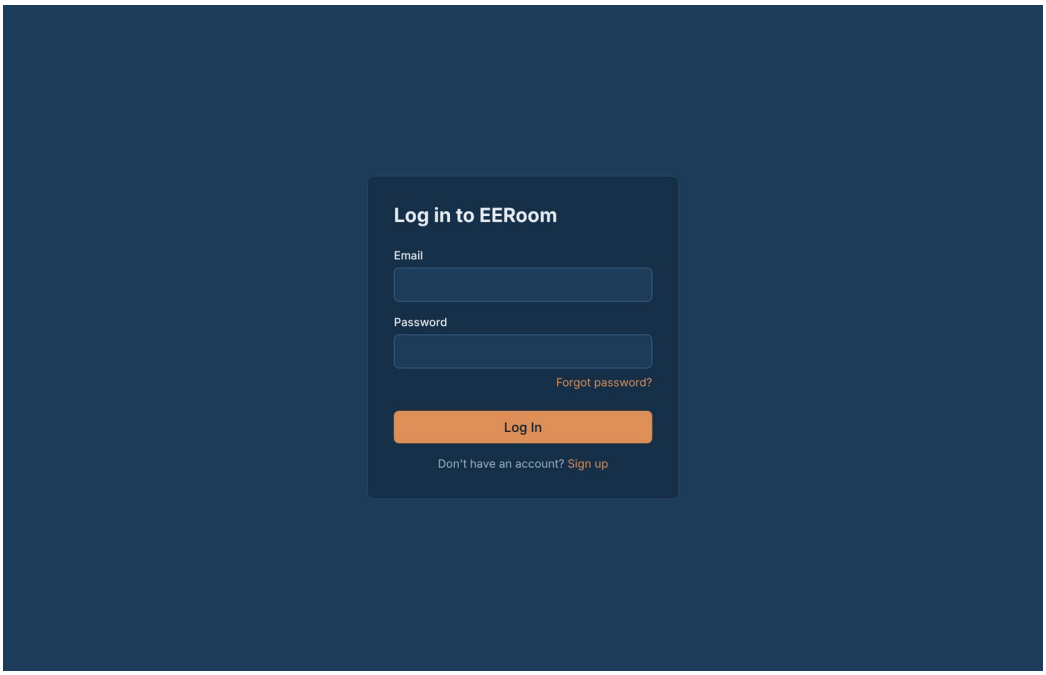
Shape buttons and numeric controls, the aliasing warning, the oscilloscope display with ideal and sampled traces, and RMS, peak-to-peak, and period readouts.



Waveform Visualizer

4.9.6 Login and Password Reset

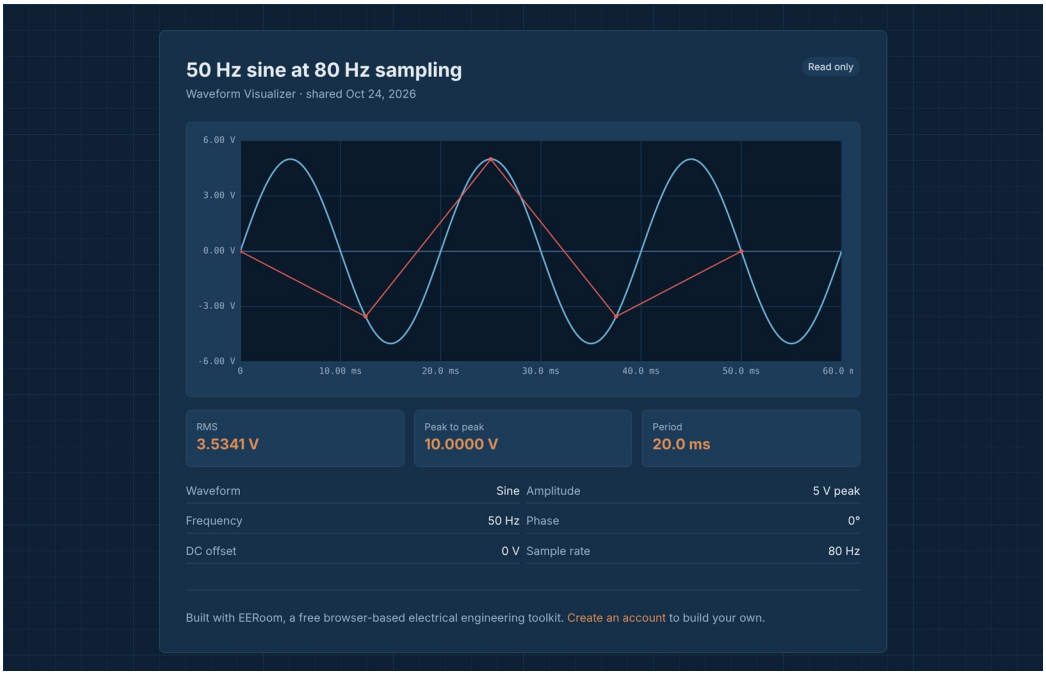
Login with a Forgot password? link. The Forgot Password page takes an email; the Reset Password page takes the new password twice.



Login and Password Reset

4.9.7 Shared Project View

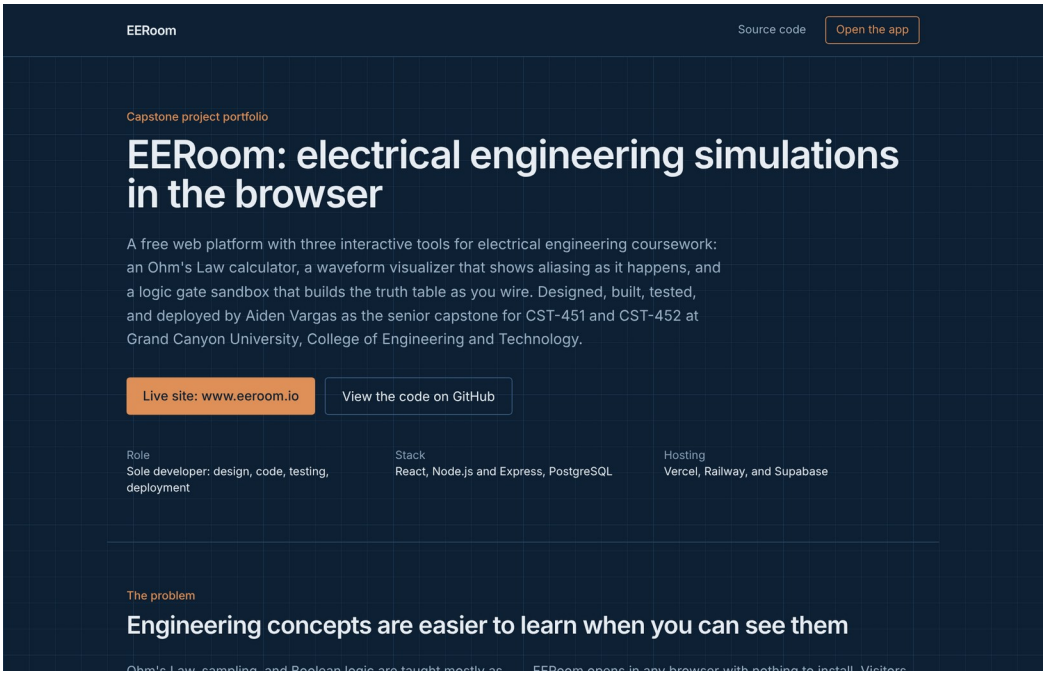
Read-only rendering of any tool's project for a visitor with no account, with a link to sign up.



Shared Project View

4.9.8 Project Portfolio

Public overview of the project for employers at /portfolio.



Project Portfolio

4.10 Security Design

Threat	Control	Where
Stolen or weak password storage	Supabase Auth hashes passwords with bcrypt; EERoom never stores them	Supabase
Password guessing	10 requests per 15 minutes per address on /auth; trust proxy set so the real address is used	authRoutes.js, app.js
Account enumeration	Login gives one vague message; reset request gives the same answer for every email	authController.js
Forged or expired sessions	JWT signed with JWT_SECRET, 24-hour expiry, checked on every /projects route	authMiddleware.js
Access to another user's data	Every project query filters by the user ID from the token; IDs in request bodies are ignored	projectController.js
Guessable share links	64-character random token; public view returns no user or project IDs	projectController.js, shareRoutes.js
Reset token leakage	Token read from the URL fragment and cleared from the address bar; verified with Supabase before use	ResetPasswordPage.jsx, authController.js
Untrusted origins	CORS limited to the origins in CLIENT_URL	app.js
Leaked secrets	Real .env files ignored by Git; database protected by Row Level Security	.gitignore, schema
Data in transit	HTTPS on Vercel, Railway, and Supabase	Hosting

These controls are tested by IT-05, IT-10, IT-13, IT-16, IT-18 to IT-20, IT-24, IT-27, IT-29, IT-31, IT-34, and IT-35, described in the Milestone 5 Coding and Testing document.

5. Data Dictionary

5.1 Table Columns

Table.Column	Type	Rules	Meaning
users.user_id	UUID	Primary key	Same ID as the Supabase Auth user
users.email	VARCHAR(255)	Unique, required	Login email
users.password_hash	VARCHAR(255)	Required	Fixed placeholder; Supabase Auth holds the real hash
users.full_name	VARCHAR(100)	Optional	Name given at registration
users.created_at	TIMESTAMPTZ	Default now	When the account was created
projects.project_id	UUID	Primary key	Project identifier
projects.user_id	UUID	Foreign key, cascade delete	Owner
projects.name	VARCHAR(100)	Required	Project name; tools fall back to an Untitled name
projects.tool_type	VARCHAR(20)	ohm, logic, or wave	Which tool opens it
projects.project_data	JSONB	Required	The tool's saved state (5.2)
projects.share_token	VARCHAR(64)	Unique, optional	Token in the share link
projects.is_shared	BOOLEAN	Default false	Whether the share link works
projects.created_at, updated_at	TIMESTAMPTZ	Default now	Created and last saved; the dashboard sorts by updated_at

5.2 project_data by Tool

tool_type	Fields
ohm	voltage, current, resistance, power (numbers or null)
wave	type (sine, square, sawtooth, triangle), amplitude, frequency, phase, offset, cycles, sampleRate
logic	nodes [{ id, type, label?, col, row }], edges [{ from, to, toPort }], inputValues { nodeId: boolean }

5.3 Client-Side Storage

Key	Storage	Contents
eeroom_token	localStorage	The 24-hour JWT
eeroom_user	localStorage	{ id, email } of the logged-in user
eeroom_pending_draft	sessionStorage	A guest's { toolType, name, projectData } until login

6. Requirements Traceability

Each in-scope requirement maps to the section of this plan where it is designed and the module that implements it. The full matrix with test cases is in the Milestone 5 Coding and Testing document.

Requirement	Design Section	Implementing Module
FR-01, FR-03, FR-05 Accounts	4.6, 4.7.1, 4.9.6	authController, useAuth, LoginPage, RegisterPage
FR-04 Password reset	4.6, 4.7.3, 4.9.6	authController, ForgotPasswordPage, ResetPasswordPage, resetLink
FR-12 to FR-16 Ohm's Law	4.8.1, 4.9.3	OhmToolPage
FR-17 to FR-21, FR-38, FR-39 Logic	4.8.2, 4.9.4	LogicToolPage, logicEvaluator
FR-22 to FR-26, FR-37 Waveform	4.8.3, 4.9.5	WaveToolPage, WaveformCanvas, waveform
FR-27 to FR-29 Projects	4.5, 4.6, 4.7.2, 4.9.2	projectController, projects API, DashboardPage
FR-30, FR-31 Sharing	4.6, 4.7.4, 4.8.4, 4.9.7	projectController, shareRoutes, ShareButton, SharedProjectPage
SC-06 Guest access	3.2, 4.7.1, 4.8.5	GuestSavePrompt, pendingDraft, LoginPage
NFR-04 to NFR-06, NFR-13 Security	4.10	authMiddleware, authRoutes, projectController
NFR-08, NFR-09, NFR-15 Tablets and accessibility	4.9.4	LogicToolPage, theme.js
NFR-14 Automated tests	1 (TR-13)	client/src/utils/*.test.js, server/tests/*.test.js

Appendix A: References

Express. (2026). Express 4.x API reference. https://expressjs.com/en/4x/api.html

Grand Canyon University. (2026). CST-452 capstone project handbook.

Node.js. (2026). Crypto: crypto.randomBytes. https://nodejs.org/api/crypto.html

React. (2026). React reference. https://react.dev/reference/react

Supabase. (2026). JavaScript client library. https://supabase.com/docs/reference/javascript

Vercel. (2026). Vite on Vercel. https://vercel.com/docs/frameworks/frontend/vite

World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/